

Decision Tree

Prerequisites:

To start with Decision Tree, you must have an understanding of:

- Probability.
- Differences between classification and regression.
- Evaluation metrics for classification and regression.

Learning Objective:

- Explain decision tree.
- Discuss the criteria for splitting a node.
- Describe how a decision tree works.
- Apply decision tree to solve classification and regression tasks in Python with scikit-learn.

Decision Tree

A decision tree is a tree of decisions. It is built on the idea that a complex problem can be solved by dividing it into several parts and solving them. For a given input data, a decision tree estimates an unknown property of the data y by asking a series of questions about its known properties x . Like an artificial neural network (ANN), a decision tree can be used for both classification and regression tasks. Even though decision trees have been around for a long time, they are still widely used today. Let's discuss more about decision tree with a simple **Fish classification** example.

Let's say, we want a model that classifies whether an animal is a fish or not by checking whether it lives in water or not. For an animal to be a fish, it should live only inside the water and shouldn't be able to sustain outside water. At first, we test whether an animal lives inside water or not i.e the attribute **Lives in Water**. All instances or animals that do not live in water are sent to the left and those that live in water are sent to the right. Thus, each such test leads to the partitioning of data. If the animals don't live in water, we label them as **Not a Fish**.

For the animals that live in water, we perform another test, **Can Live outside Water**. The animals which cannot live outside water are sent left and we label them as **Fish** whereas those that can live outside water are sent to the right and we label them as **Not a Fish**.

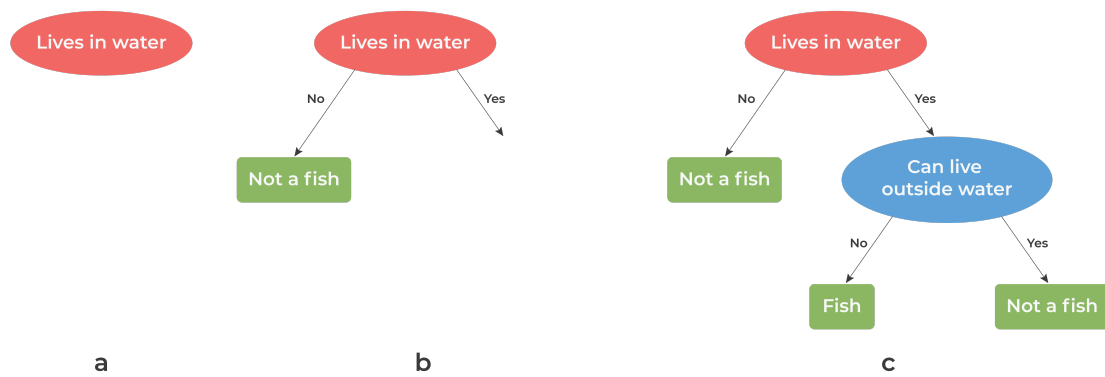


Figure 1: a) Root Node b) Root node with leaf node c) Complete Decision Tree

Each test that we performed is represented as a **node** in the figure above. The node in which we perform the first test, **Lives in Water**, is called the **root node**. Just like ours', most problems will require more than one test. Nodes, that perform a test but are not the root node, in our case **Can Live outside Water**, are called **internal nodes**. There can be multiple internal nodes. As we increase the tests, our model grows and forms a tree-like structure and hence the name **decision tree**. Like biological trees, a decision tree has leaves termed as **leaf nodes**. Leaf nodes in a decision tree assign labels or give a final decision. In our case *Fish* and *Not a Fish* are the leaf nodes. In addition to nodes, a decision tree has a property called **depth** which is the maximum number of edges between root and leaf node. Depth can also be defined as the total number of tests required to reach a leaf node.

In the figure above, node colored red is the root node (also called the parent node), node colored blue is an internal node (child node of the root node), and nodes colored light-green are leaf nodes.

Impurity Metrics and Selection of the Best Attribute

The decision tree above used two attributes: **Lives in water** and **Can live outside water** as test/split functions. In real life, we may not always be lucky to get such data. Most real datasets have multiple attributes thus leading to confusion about which attribute to choose as a test/split function at each node. So, to identify the best attribute for splitting at each node, we use metrics like **gini**, **entropy**, and **mean squared error (MSE)**. These metrics are called **impurity metrics**.

Let us try to understand the impurity metrics and how they help to determine which attribute to use to split by using a toy classification problem with three classes and two features shown below. The codes to generate these plots are placed in a separate notebook. You can check the codes if you are interested.

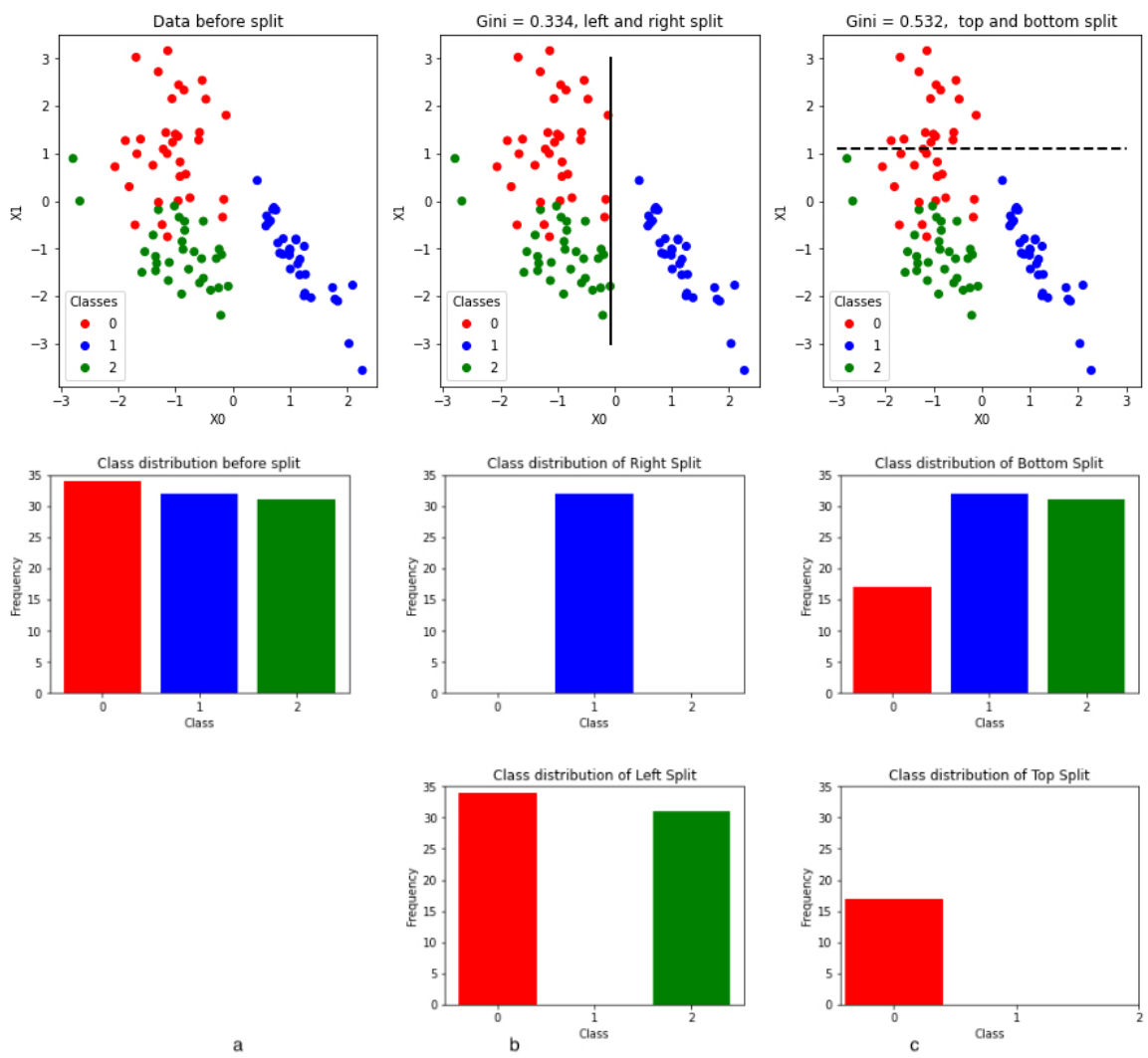


Figure 2: Test attributes and their associated gini values. a) Dataset at the parent node before split. b) After a vertical split. c) After a horizontal split.

The leftmost figure shows the scatter plot and the class histogram of the training samples in a parent node. Our objective is to select the best attribute (amongst x_0 and x_1) that separates the different classes as much as possible. When splitting the data vertically using x_0 (the center plot), we obtain two sets of data. In one set (right set), we obtain data from only one class only (pure set) while, the other set (left set) contains data from two classes (impure set). Similarly, when splitting the data horizontally using x_1 , the top set contains data from one class only (pure set) while the bottom set contains data from all three classes. You can also notice the gini values at the top of the plots after the vertical and horizontal splits. Smaller values of impurity metrics mean better split. Thus, for this particular node, we select the vertical split or x_0 attribute as it reduces the uncertainty of prediction better than the horizontal split or x_1 attribute.

Let's go back to the **Fish classification** example above. Can you verify that using **Can Live outside Water** test in the root node would have reduced the uncertainty in prediction better than the attribute **Lives in Water**? The decision tree above would have only one node! Such a tree with a single node is called a **decision stump**.

Now we know about the impurity metrics, let's delve into them. Throughout this reading material, we will be talking about the gini impurity. Mathematically, gini is given by:

$$\text{Gini} = 1 - \sum_{i=1}^n P_i^2 \quad (1)$$

where P_i is the probability of an object being classified to a class i . Now, let's write our function to compute the gini impurity using `numpy`.

```
In [ ]: import numpy as np

def compute_gini(class_frequencies):
    '''Takes in class distribution, returns gini impurity'''
    class_frequencies = np.array(class_frequencies)
    probability = class_frequencies/sum(class_frequencies)
    gini = 1- sum(probability**2)

    return gini
```

Let's suppose that we have four different samples of data for the **Fish classification** example above. We will use the function `compute_gini()` to compute the gini impurities.

Fish	Not a Fish	Remarks
10	0	Pure Data
1	9	Unevenly Distributed Impure Data
7	3	Unevenly Distributed Impure Data
5	5	Evenly Distributed Impure Data

```
In [ ]: # Computing gini impurities for different proportions of data shown in the table above

class_dist_matrix = np.array([[10,0],[1,9],[7,3],[5,5]])

for num_cases in range(len(class_dist_matrix)):
    class_dist = class_dist_matrix[num_cases,:]
    gini_impurity = compute_gini(class_dist)
    print("{:.2f}".format(gini_impurity))

0.00
0.18
0.42
0.50
```

From the computations above, we can see that a pure set (samples from one class only) has zero impurity. Impurity (uncertainty of prediction) becomes maximum when a set has an equal number of samples from each class. Just like gini and entropy, MSE also measures impurity in data. It is used in regression trees. Attribute that yields split with lower MSE are preferred over a split with higher MSE.

Decision Tree Algorithm

In the previous sections, we learned basic concepts about decision tree—its structure, the function of different types of nodes, and impurity metrics. Now let's see how these pieces fit together to construct a decision tree.

1. If all the records in the set belong to the same class, label it as a leaf node and stop.
2. If the set contains records that belong to more than one class,
 - Select the best attribute to split using the impurity metric (gini, entropy, MSE).
 - Split the data into smaller subsets, child nodes, depending upon the value of the attribute selected above.
 - For each subset, repeat step 1.

Note : Here, we are using a single stopping criterion i.e. if all instances belong to the same class, we label it as a leaf node and stop splitting. However, there exist other stopping criteria that are left for future study.

Decision tree in scikit learn

To construct a decision tree, scikit-learn provides two classes [DecisionTreeClassifier](#) for classification, and [DecisionTreeRegressor](#) for regression. In this reading material we will be focusing on classification and in the assignment we will get to implement `DecisionTreeRegressor` for solving a regression problem. In our classification problem, we will be using the **iris dataset** by **Ronald Fisher** published in **1936** and available at [UCI Machine Learning repository](#) under [CC0: Public Domain](#). Details of the dataset are provided in the following section.

`DecisionTreeClassifier` provides us with the following functions to help us construct the decision tree:

- `fit(X, y)` - For training the decision tree with our data `X` as the features and `y` as the labels.
- `predict(X)` - For predicting the labels of unknown data `X`.

Apart, from these, we will also be using functions

- `train_test_split()` - To split the data into training and testing set.
- `load_iris()` - To load iris dataset from [datasets](#) module in `scikit-learn`

Dataset Description:

Iris Data Set

The iris dataset contains 150 samples of 3 variants of the iris flower. The dataset has 4 features (sepal length, sepal width, petal length, petal width) and a class label with values 0 or 1 or 2 denoting the type, Setosa, Versicolor, and Virginica respectively.

Imports

```
In [ ]: from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import datasets
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree
from matplotlib.colors import ListedColormap
```

Now, let's use [DecisionTreeClassifier](#) to fit a decision tree on the training data and use it to get the

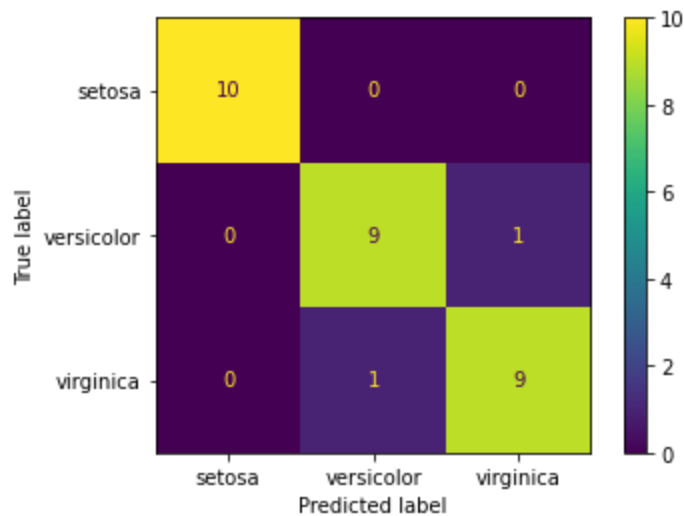
predictions for the test data.

```
In [ ]: # Loading the dataset and fitting a decision tree
iris = datasets.load_iris()
X, y = iris.data, iris.target
class_names = iris.target_names

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42) #Trees grow to a maximum
tree_clf.fit(X_train, y_train)

# Plotting confusion matrix
disp = ConfusionMatrixDisplay.from_estimator(tree_clf, X_test, y_test, display_labels=class_names)
plt.show()
```

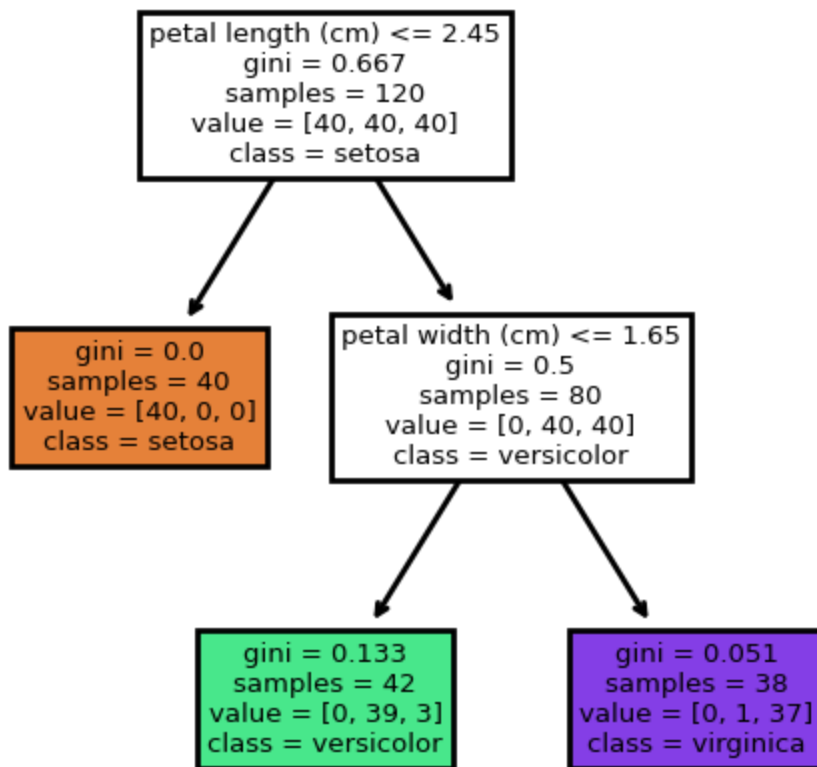


From the confusion matrix above, we can see that the decision tree correctly classifies almost all the test instances. Specifically, class 0 (Setosa) is perfectly predicted. This is because the instances of Setosa form a single cluster (concentrated region), and don't mix with other classes. This is shown in the decision surface plot later on.

Now we have trained the decision tree, let's visualize its structure. For this purpose, we use scikit-learn's [plot_tree](#) function.

```
In [ ]: # Decision tree visualization

fig, axes = plt.subplots(nrows = 1, ncols = 1, figsize = (3, 3), dpi=200)
axes.plot(linewidth = 0)
_ = plot_tree(tree_clf,
              feature_names = iris.feature_names,
              class_names=iris.target_names,
              filled = True,)
```



Let's peek into the information provided by the nodes of the tree. For now, we will consider the *leaf node with a green background*. However, the information provided by other nodes can be interpreted similarly. The left edge indicates 'True' while the right edge indicates 'False'.

gini represents the gini impurity associated with the node.

samples is the total number of instances that reached this node. It is the sum of all elements in the **value** list.

value is a list that represents instances from each class that reached this particular node during the training phase. In this node, `value = [0, 39, 3]` means 0 instances of Setosa, 39 instances of Versicolor, and 3 instances of Virginica reached this node.

class represents the class/label the decision tree would assign an instance reaching this node. This node has **class** Versicolor because most of the instances in this node are from Versicolor class.

You can also notice that the gini value decreases as the depth increases.

Note: In `scikit-learn`, if the **value** has an equal number of instances from two or more classes, the **class** will be assigned the label which comes earlier when sorted alphabetically. Can you verify this with the root node and the internal node?

Visualizing the decision boundary

A decision tree performs recursive partitioning of the original data until it meets a stopping criterion (*depth* in our example above). The decision tree above has two nodes with test (split) functions. Each test results in an axis-parallel decision boundary. The first (root) node tests `petal length (cm) <= 2.45` while its

child node (internal node) tests `petal width (cm) <= 1.65` . Both these split results into subsets that are purer (check the gini values) than the set in the root node. Below, we visualize the decision boundary and surface generated by each test function. The decision surface or the classification map allows us to visualize the regions of space that correspond to different classes. For example, all the samples that fall under the red region, will be labeled class 0 and so on.

```
In [ ]: # taking only the features petal width and petal length for easier visualization
X = iris.data[:, [2, 3]]
y = iris.target

# Training a decision tree
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X, y)
```

```
In [ ]: #@title Visualization code (utils.py)

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

def decision_surface_visualization(X, y, tree_clf):
    plot_step = 0.02
    fig, ax = plt.subplots()
    fig.set_figheight(7)
    fig.set_figwidth(7)

    # For decision surface
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step),
                        np.arange(y_min, y_max, plot_step))
    colors = ListedColormap(['r', 'g', 'b'])

    Z = tree_clf.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    cs = plt.contourf(xx, yy, Z, cmap=colors)

    # Decision boundary plotting ends

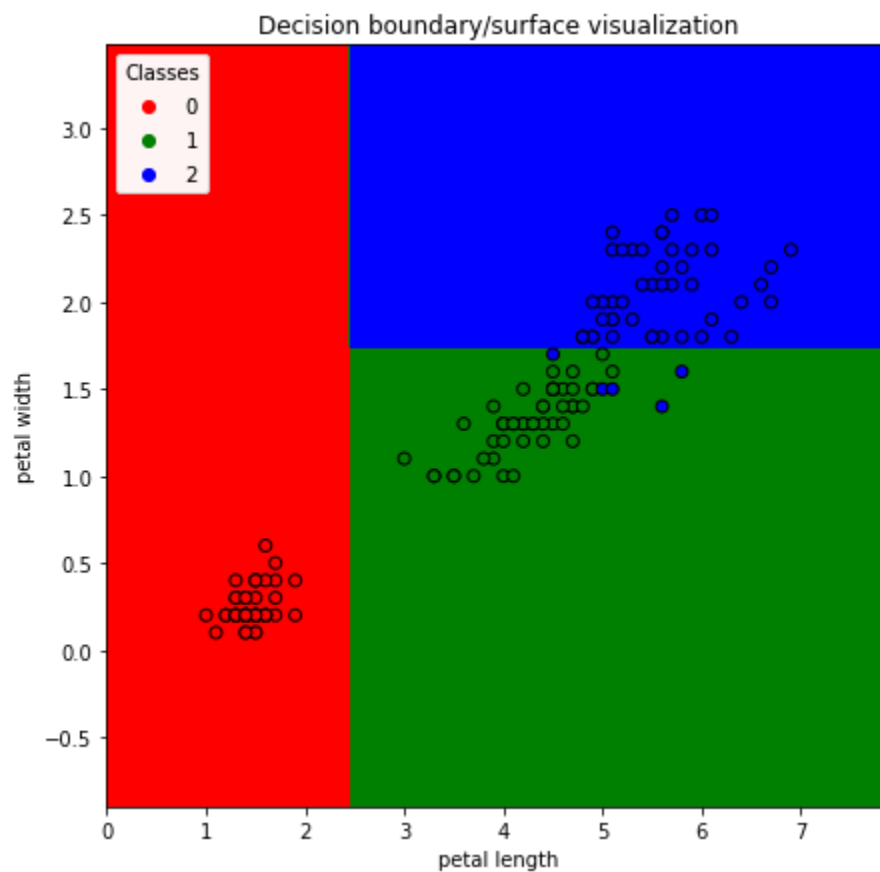
    scatter = ax.scatter(x=X[:, 0], y=X[:, 1], c=y, edgecolor='black', cmap=colors)

    # Produce a legend with the unique colors from the scatter
    legend1 = ax.legend(*scatter.legend_elements(),
                      loc="upper left", title="Classes")
    ax.add_artist(legend1)

    # Produce a legend with a cross section of sizes from the scatter
    handles, labels = scatter.legend_elements(prop="sizes", alpha=0.6)

    plt.xlabel('petal length')
    plt.ylabel('petal width')
    plt.title("Decision boundary/surface visualization")
    plt.show()
```

```
In [ ]: ##from utils import decision_surface_visualization
decision_surface_visualization(X, y, tree_clf)
```

Note: Here class 0, 1 and 2 represent Setosa, Versicolor and Virginica respectively.

Additional Resources

Visualizing Decision Trees:

- Terence Parr and Prince Grover, How to visualize Decision Trees, <https://explained.ai/decision-tree-viz/>

Decision Tree Theory

- Pang-Ning Tan, Michael Steinbach, Anuj Karpatne, Vipin Kumar, [Introduction to Data Mining](#), 2nd Edition
 - Check unit 3.3.1, 3.3.2, and 3.3.3 page 119 to understand how decision tree work and how to build them
- A. Criminisi and J. Shotton, Decision Forests for Computer Vision and Medical Image Analysis
 - Check chapter 3 page 7 to get a basic understanding of tree—data structure and decision tree.